

Horizon-Gated Latent Prediction: Disentangling Persistent Factors in Time Series

Jaechan Lee
University of Waterloo
Waterloo, Canada
j2342lee@uwaterloo.ca

Abstract—Time-series representation learning has advanced considerably, but a single series mixes factors acting on different timescales, which makes the embedding hard to interpret and use. Work that separates them at the embedding level has mostly addressed static factors, those that do not change over an entire sequence, because invariance by definition supplies a training signal. Persistent factors that vary slowly without being constant get no such free constraint, and they need an axis that fixes where the boundary between slow and fast lies. That axis is already present in prediction-based representation learning: the horizon sets how far into the future the model must predict, and therefore how long information must persist in order to be useful. We propose Horizon-Gated Latent Prediction (HGLP), which places a single threshold on that axis. Beyond it the gate forces the predictor to use only a designated subspace, z_{slow} , while a cross-covariance penalty between the blocks holds the two apart. A predictor cannot foresee upcoming transitions, so the only way to lower error at long horizons is to record the current state. We also propose a separation index (SEP) that scores how far apart the two blocks are. On synthetic and real data in which persistent and transient factors coexist, we confirm that the transient factor can be separated out of z_{slow} . Against post-hoc rotations of an ungated embedding, we lead on the synthetic data but trail Slow Feature Analysis on the real ones. We further quantify the downstream trade-off that the separated embedding incurs, and report that it gives up downstream accuracy while degrading less in a new domain when labels are scarce.

Index Terms—time-series representation learning, self-supervised learning, disentanglement, joint-embedding predictive architecture, interpretability

I. Introduction

Time-series representation learning has advanced on top of predictive objectives — contrastive prediction of the future [1], and more recently prediction of latent representations rather than reconstruction of the raw signal [2], [3]. A single series, however, mixes factors acting on different timescales, which makes the embedding hard to interpret. Work that pulls them apart has succeeded mainly at separating static factors, those constant over an entire sequence [4], [5], because invariance is itself a training signal that can be written into the objective. For factors that change slowly without being constant, no such definition is available.

Separating what is “slow” from what is “fast” can mean two different things. One is separating the signal — decomposing the observed waveform into a trend and a fluctuation, for which long-established tools exist [6],

[7]. The other is separating the factors that generated the signal, the lineage of latent-factor identification from temporal structure [8]. This work is concerned with the factor side — a patient’s diagnosis, or the fact that a person is walking. The two readings are often treated as the same, yet they need not agree.

They can diverge in only one direction. We fix the term: a factor is persistent if it still contributes to prediction Δ steps ahead. Under this definition the slow part of the signal is necessarily persistent, since a component confined to low frequencies has slowly decaying autocorrelation [9], and remaining autocorrelation is a contribution to prediction at distant horizons. The converse fails: low-pass filtering is linear in the signal, whereas a factor may ride on nonlinear statistics such as frequency or variance. Fig. 1 shows such a case on controlled data: when the persistent factor sets the frequency of the oscillation, reading only the slowly varying part of the signal recovers the transient factor ($R^2 = 0.90$) and not the persistent one ($R^2 < 10^{-5}$).

The two readings are still connected. While a factor persists, the statistics of the signal are fixed: during walking the acceleration waveform oscillates rapidly, but its variance and spectral shape stay constant. What changes slowly is not the signal but its statistics, and those appear only through some function of the signal. In Fig. 1 that function is the oscillation frequency, read over a wide window (Fig. 1b); with a narrow window the reading is mostly the transient factor and turns over toward the persistent one only once the window is tens of times the transient lifetime (Fig. 1c).

Which band a factor appears in therefore constrains its persistence in one direction only: the slow band implies persistence, not the converse. This is an existence claim — such factors can exist; how often they arise in real data this paper does not answer. Our selection criterion is persistence. In our synthetic data the persistent factor is carried inside the oscillation, the fastest-changing part of the signal, while the transient factor enters as a slower additive term.

Persistence is controlled by a variable prediction-based learning already has: the horizon Δ . Setting Δ farther out makes only long-persisting factors useful [10], [11]. We gate that horizon and propose HGLP, which forces prediction at distant horizons to use only a designated

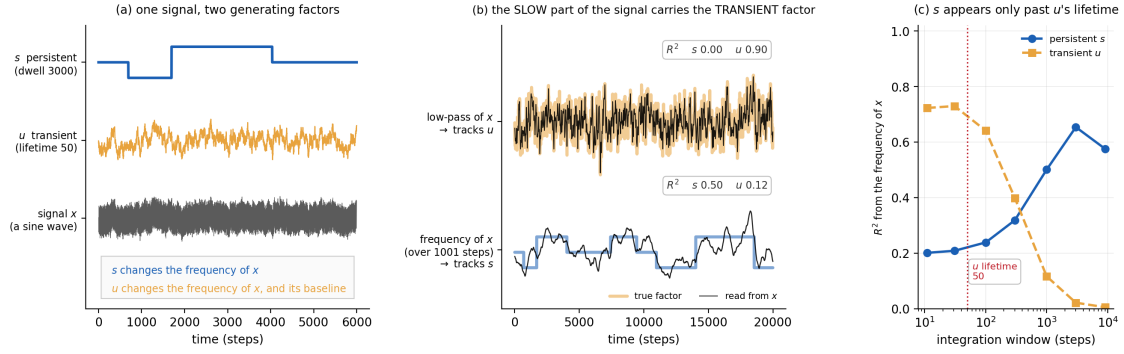
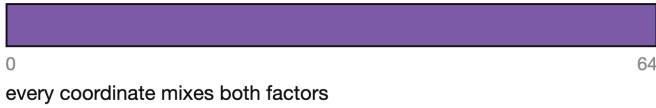


Fig. 1. Spectral slowness is not factor persistence. One synthetic series (400,000 steps, a single seed): the persistent factor s switches every 3,000 steps among three frequencies 3% apart, and the transient factor u is an Ornstein–Uhlenbeck process of lifetime 50. Autocorrelation half-lives are 1,859 and 51 steps, so the two timescales are well separated. (a) The two factors and the signal they generate. s changes only the frequency of x ; u changes its frequency and its baseline. (b) Reading the slow part of x recovers the transient factor rather than the persistent one (R^2 0.90 against 0.00), while reading the frequency of x over a 1,001-step window recovers the persistent one (R^2 0.50). (c) R^2 for both factors against the width of that window. The persistent factor appears only once the window is tens of times u ’s lifetime, and falls off again when the window grows wide enough to blur s itself.

Ordinary embedding



What we ask for



what a coordinate carries

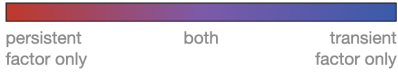


Fig. 2. What we ask of the embedding. In an ordinary embedding every one of the 64 coordinates mixes both factors. We ask instead that a nameable 16-dimensional block, z_{slow} , be emptied of the transient factor; the remaining block z_{mix} is left unconstrained and keeps both. The claim is about what is absent from z_{slow} , not about where the persistent factor can be found — the latent space is rotation-symmetric, so any 16 coordinates carry it.

block of coordinates. The target state is shown in Fig. 2: an ordinary embedding mixes both factors in every coordinate, whereas we require that a nameable 16 dimensions (the z_{slow} block) must carry the persistent factor with the transient factor emptied out of it. The latent space is rotation-symmetric, so the persistent factor can be read out of any 16 dimensions; our claim is therefore not that “the persistent factor gathers in z_{slow} and is absent elsewhere,” but that “the transient factor is emptied out of z_{slow} .” The remaining block (z_{mix}) is unconstrained and holds both factors.

Our contributions are three. (i) An inductive bias that gates an already existing axis, the prediction horizon, splitting the persistent factor into a designatable block. (ii) An evaluation procedure that measures that separation with a block $\times$ factor matrix rather than from one side

only, together with a single summarizing index, SEP. (iii) Measurements on synthetic data, PTB-XL, and HAPT. The results cut both ways. On synthetic data designed so that persistence and slowness disagree, we lead label-free post-hoc rotation on both stems (SEP 0.849 against SFA 0.388); on real data we trail SFA, which leads in three of four settings by margins of 0.032 to 0.072. We read this as a consequence of the relation between persistence and slowness — on real data the two mostly coincide, so optimizing slowness directly does the same job more directly — though we did not measure that causal link. Separation also carries a downstream cost (macro-F1 -0.111 on HAPT), which we quantify.

II. Related Work

We group the related work into three strands: classical methods, which fix a transform and read the signal through it; self-supervised learning for time series, which learns representations without labels; and disentangled representations for time series, which attaches meaning to the learned coordinates.

A. Classical Methods

Classical methods apply a fixed transform to the raw signal to read out its factors, and the dominant line has been the search for slowly varying components, since the slowly varying part of a signal reflects the factors that persist. The most direct approach splits the waveform itself: STL decomposes a series into seasonal and trend components [6], and EMD extracts oscillatory modes empirically [7]. These deliver components of the signal, not the factors that generated it.

Another strand optimizes the slowness of the output directly. Slow Feature Analysis (SFA) minimizes the temporal derivative of the output under unit variance and pairwise decorrelation constraints, and, since the input is expanded into low-order polynomials, it also extracts nonlinear features [12]. Strengthening decorrelation into

statistical independence makes the slow components separate from one another too, combining slowness with independent component analysis [13]. We use these two variants as our label-free post-hoc rotation baselines. A further strand changes the criterion: Predictable Feature Analysis (PFA) keeps SFA’s extraction scheme but selects the most predictable features instead of the slowest, predictability being how well the next value is approximated from the most recent p values [14]. Yet another changes where the signal is read: the Fourier transform reaches factors riding on the modulation rather than the values, and in rotating-machinery diagnosis a bearing fault’s characteristic frequencies appear not in the raw spectrum but in the spectrum of the envelope obtained through the Hilbert transform [15].

What this family has in common is that the transform is not learned: which statistics and bands to keep are fixed in advance by a human, and the projection is refitted over the whole signal whenever the domain changes.

B. Self-Supervised Learning for Time Series

Time-series representation learning trains an encoder mapping a signal segment to a vector without labels, taking its learning signal from the temporal structure of the data—neighboring segments should be similar, or what is observed now should predict what comes shortly after. The family splits into three groups by the form of the target.

The reconstruction family is the largest and effectively the default. Autoencoders pass a signal through a narrow bottleneck and restore it, the remaining error itself being useful for anomaly detection, imputation, and forecasting pretraining. Variational autoencoders add a prior over the latent variables, and that prior becomes a handle for inducing structure in the latent space: a representative variant weights the KL term to push the coordinates toward independence [16], a lineage leading to Section II-C. Masked pretraining, which reconstructs randomly masked patches, has since become established [17].

The contrastive family decides which pairs count as similar and pulls and pushes the representations accordingly. CPC predicts the future contrastively in the latent space [1], and later time-series methods refined those pairs: contrasting hierarchically over augmented context views [18], or using local smoothness to define stationary temporal neighborhoods [19]. Their weakness is that what gets discarded is hidden in the pair-selection rule.

The latent-prediction family takes the representation itself, not the raw signal, as the target [2], [3]. Its time-series variants differ in where the target is read: from masked patches [20], from the next latent token [21], or from an attention-pooled summary of the whole future interval [22].

This work belongs to the latent-prediction family but uses both a regression-form (L_1) and a contrastive-form (InfoNCE) loss, reporting the two stems together to show

that the horizon gate does not rely on a particular loss form. All three groups learn the representation, but the objective does not determine what each coordinate means.

C. Disentangled Representations for Time Series

Work here has been most successful at separating factors constant over an entire sequence from the rest, because invariance transfers directly into an objective. FHVAE separates what stays fixed within a sequence from what changes across segments by assigning different priors [4], and the disentangled sequential autoencoder splits the latent into static and dynamic parts [5]. On identifiability, exploiting non-stationarity by learning to distinguish time segments makes a nonlinear ICA model identifiable up to a pointwise transformation of each factor when combined with linear ICA [8], and an approach placing a sparse prior on the change between adjacent observations followed [23]. That models cannot be selected without labels [24] applies equally to our hyperparameters, and we address it in the limitations.

What this family leaves empty is factors that vary slowly but are not constant: the static/non-static dichotomy pushes them into the “dynamic” side and separates them no further. None of the three strands creates coordinate-block structure of the kind we seek; this work fills that place with the axis of the prediction horizon.

III. Data and Methods

A. Data and Factors

We use one synthetic dataset of our own design and two public real-world sets (Table I). In each, the persistent factor is the quantity we try to separate and the transient proxy is a fast-varying quantity computed directly from the signal, not from labels.

1) Synthetic data: A three-state Markov persistent factor s (mean dwell 3,000 steps) sets the carrier frequency of an oscillation, spaced $\pm 3\%$ around 0.100; a transient Ornstein–Uhlenbeck factor u (autocorrelation time 50 steps) modulates that instantaneous frequency and adds to the signal. Amplitude is identical across regimes, so a regime change alters only how fast the signal oscillates. Consequently the persistent factor does not appear in the slow component: a low-pass reading ($f < 0.02$) recovers the transient factor ($R^2 = 0.90$) and essentially not the persistent one ($R^2 < 10^{-5}$; Fig. 1).

Difficulty is set by the spacing between regime frequencies, calibrated by a training-free FFT reader. At $\pm 3\%$ it reaches accuracy 0.660 (chance 0.333); at $\pm 5\%$ it is already 0.784 before any training, and at $\pm 1\%$ it falls to 0.459, near chance. We therefore use $\pm 3\%$ throughout.

2) PTB-XL: PTB-XL holds clinical 12-lead electrocardiograms, each 10 seconds long; we use lead II. Our subset is 5,000 records at 100 Hz from 4,812 patients, classified as normal or abnormal with balanced classes, so chance is 0.500. The persistent factor is the diagnosis, constant within a record; the transient proxy is the instantaneous

TABLE I. The three datasets. Persistent factor = the factor we try to separate; transient proxy = a fast-varying quantity computed directly from the signal, not from labels. Chance level is $1/k$ for balanced macro-F1.

	Synthetic	PTB-XL	HAPT
Samples	10^6 steps	5,000 records (4,812 patients)	2,104 windows (30 subjects)
Sampling rate	—	100 Hz	50 Hz
Channels	1	1 (lead II)	3 (accel. x, y, z)
Patch	8 steps	10 samples (0.1 s)	4 samples (0.08 s)
Window length L	256 patches (2,048 steps)	100 patches (10 s)	256 patches (20.5 s)
Persistent factor	oscillation regime s (3 classes)	ECG diagnosis (normal/ abnormal, 2 classes, constant per record)	physical activity (6 classes)
Transient proxy	OU factor u	instantaneous voltage	accel. magnitude $\ a\ $
Chance level	0.333	0.500	0.167

voltage. We cut into 100 patches of 10 samples (0.1 s), normalize with per-channel statistics from the training split alone, and split per patient. This is the simplest form of our setting, a label constant over the whole record, but it strains the assumption our threshold rule relies on — that the transient factor has a decaying autocorrelation from which a lifetime can be read. The heartbeat is a regularly repeating component, so its autocorrelation oscillates instead of decaying, and the threshold has to be set by hand (Section III-F).

3) HAPT: In HAPT (Human Activities and Postural Transitions), thirty subjects wearing a waist-mounted smartphone performed six activities — walking, walking upstairs, walking downstairs, sitting, standing, lying — with postural transitions as separate classes. We use the three acceleration axes at 50 Hz and not the gyroscope. The persistent factor is the activity being performed, scored only over the six basic activities; the transient proxy is the acceleration magnitude $\|a\|$. Transition and unlabeled segments (24.7% of the total) are excluded from scoring but not removed from the pre-training input, since selecting training data by labels would undermine the claim that training uses no labels. Windows are 256 patches of 4 samples (0.08 s), i.e. 20.5 s, giving 2,104 windows; activity boundaries are not consulted when cutting, so labels are read per position. Normalization and splitting follow PTB-XL, with the subject as unit. Unlike the fixed diagnosis in PTB-XL the activity changes within a record, so 57% of windows cross a transition and the persistent factor is not constant within a window (see Results).

B. Model

Encoder. One patch becomes one token (Fig. 3(a)) through a single linear projection; there is no codebook and no quantization. Channels are mixed at this first layer by design, since the factor of interest is sometimes defined by the relation between channels — HAPT posture is determined by how gravity distributes over the three axes. Tokens then pass through a stack of causal transformer layers with rotary position encoding, and a final linear layer produces the embedding, whose leading coordinates form z_{slow} and the remainder z_{mix} . The last operation is a per-block LayerNorm, applied to the two blocks separately

rather than to the embedding as a whole, for the reason given in Section III-D. The predictor is a small MLP that sees only the embedding at one time point together with the horizon, which enters as a learned expansion of $\log_2 \Delta$; one predictor is conditioned on the horizon rather than one head per horizon, consistent with treating the horizon as a continuous axis.

No modification is made to the attention operation: the gate sits between encoder output and predictor input, the penalty in the loss, the per-block LayerNorm in the encoder’s last line. This is a change of training procedure, not of architecture, so it attaches to any causal sequence encoder.

Target.

$$z^* = \text{sg}[f_{\bar{\theta}}(x_{[t+\Delta-w, t+\Delta]})_{-1}], \quad \bar{\theta} \leftarrow m\bar{\theta} + (1-m)\theta \quad (1)$$

The prediction target is not the raw signal but the representation an EMA copy of the encoder reads from a short window inside the future interval, with the gradient stopped. Bounding that window keeps the target’s receptive field inside $(t, t + \Delta]$, so it never re-contains the anchor’s own past and the shortcut of matching the target from what is already known at the anchor is closed by construction. The target never passes the gate and is always the full embedding: at far horizons the model is asked to match the whole future embedding from the persistent block alone, and this asymmetry is the core of the mechanism.

Loss.

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{xcov}}, \quad \mathcal{L}_{\text{pred}} \in \{\mathcal{L}_1, \mathcal{L}_{\text{NCE}}\} \quad (2)$$

The two forms share backbone, gate and penalty and differ only in the loss layer. HGLP-Reg uses the L_1 distance; L_2 ’s squared-error gradient is dominated by large residuals and buries a weak persistent signal, which made results swing across seeds. HGLP-NCE uses InfoNCE over cosine similarity with temperature 0.1, with one departure from the standard form: samples from the same window are excluded from the negative pairs, since they share the persistent factor and pushing them apart would teach the model to discard it.

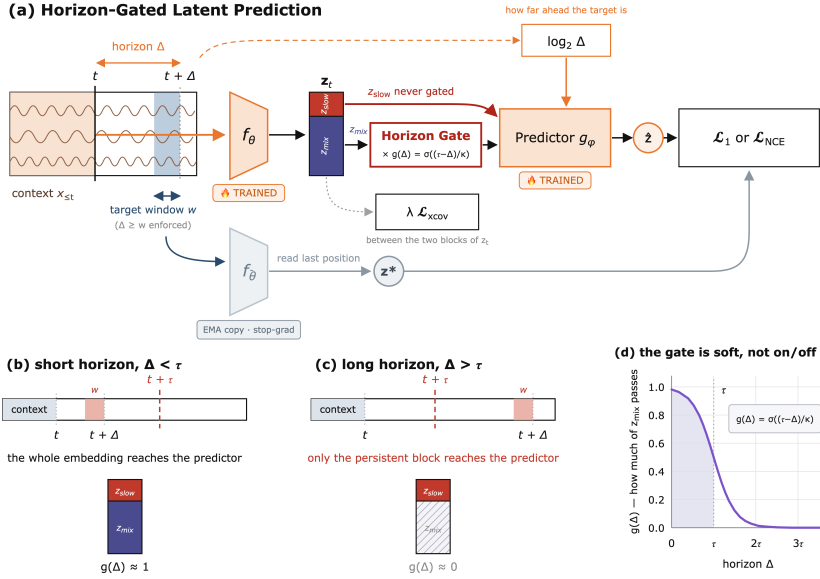


Fig. 3. Horizon-gated latent prediction. (a) A causal encoder maps a window to an embedding z_t , split into a persistent block z_{slow} (16 of 64 dimensions) and the rest z_{mix} . A predictor conditioned on the horizon Δ must match a target read at one position inside $(t, t + \Delta]$, produced by an EMA copy of the encoder. A cross-covariance penalty is applied to z_t before the gate. (b, c) The same pipeline at a short and a long horizon. z_{slow} is never gated; only z_{mix} is, and only past τ , so at long horizons the only route to lowering the loss runs through z_{slow} . (d) The gate is a sigmoid of width κ , not a switch. τ is set from the transient factor’s lifetime and Δ_{max} is set separately, so the axis is drawn in units of τ rather than absolute horizons.

C. The Horizon Gate

$$g(\Delta) = \sigma\left(\frac{\tau - \Delta}{\kappa}\right), \quad \tilde{z}_t(\Delta) = [z_t^{slow}; g(\Delta) z_t^{mix}] \quad (3)$$

Immediately before the predictor, the rear block is multiplied by $g(\Delta) \in (0, 1)$ (Fig. 3(b)–(d)): for $\Delta \ll \tau$ the embedding passes unchanged, for $\Delta \gg \tau$ the rear block is suppressed. Dimensions are multiplied, not truncated, so the predictor input is always D -dimensional; z_{slow} is never blocked; and the gate is a smooth curve, not a step. A step gate would flip horizons near τ between fully passed and fully blocked under the smallest movement of the threshold, whereas width κ gives them intermediate values. Since τ cannot be chosen accurately without labels (Section III-F), this buffer is necessary rather than convenient, and in our runs a step gate lowered the separation metric on both loss forms.

Gradient split.

$$\frac{\partial \mathcal{L}}{\partial z_t^{slow}} = \frac{\partial \mathcal{L}}{\partial \tilde{z}_t^{slow}}, \quad \frac{\partial \mathcal{L}}{\partial z_t^{mix}} = g(\Delta) \frac{\partial \mathcal{L}}{\partial \tilde{z}_t^{mix}} \quad (4)$$

Because the encoder is shared across horizons, this one line splits the training signal the two blocks receive: z_{slow} is updated at every horizon, whereas z_{mix} is updated in proportion to $g(\Delta)$ and so effectively only at near horizons. Two consequences follow, and only one of them is guaranteed. Availability is enforced: a predictor cannot foresee when the persistent factor will next switch, so at long horizons it can lower the loss only by recording the current state, and the gate leaves z_{slow} as the sole route for doing so; so whatever still contributes to prediction

that far ahead must be readable there, and no predictor however large can restore information the gate has suppressed to zero. Exclusion is not enforced: z_{slow} is still used at near horizons, where transient information lowers the loss, and no term of the objective pushes that information out of z_{slow} — exclusion has to come from the penalty of Section III-D. We therefore measure exclusion rather than assume it, and the claim is that the transient factor is emptied out of z_{slow} , not that the persistent factor gathers there: with all mechanisms absent, the first 16 dimensions and a random 16 agree to within 0.015.

D. Pulling the Two Blocks Apart

Cross-covariance penalty.

$$C = \frac{1}{N-1} \bar{Z}_{(1:d_{slow})}^\top \bar{Z}_{(d_{slow}+1:D)}, \quad \mathcal{L}_{xcov} = \frac{1}{d_{slow}(D-d_{slow})} \sum_{a,b} C_{ab}^2 \quad (5)$$

The penalty pushes the two blocks to be linearly unpredictable from each other and acts on the embedding before the gate. It travels with the per-block LayerNorm: a single LayerNorm over the whole embedding gives the two blocks one shared mean and variance, recreating exactly the coupling the penalty is trying to remove. “Mechanism-free” therefore means all three devices absent — gate, penalty and per-block LayerNorm.

E. What We Measure

Reading the persistent factor from z_{slow} is not by itself evidence of separation, since rotational symmetry means it is read about as well from any 16 dimensions. We therefore report a block $\times$ factor matrix scoring both blocks on both

factors, plus a single-number summary SEP, which asks three questions.

- 1) Inclusion — is the persistent factor in z_{slow} ? We score how well the persistent factor is predicted from z_{slow} alone.
- 2) Allocation — is the transient factor in z_{mix} ? We score how well the transient proxy is recovered from z_{mix} alone.
- 3) Exclusion — is the transient factor absent from z_{slow} ? We measure how well the transient proxy can be recovered from z_{slow} , and subtract that value from 1.

$$\text{SEP} = \underbrace{[s_{\text{per}}(z_{\text{slow}})]_0^1}_{\text{inclusion}} \cdot \underbrace{[s_{\text{tra}}(z_{\text{mix}})]_0^1}_{\text{allocation}} \cdot \underbrace{[1 - s_{\text{tra}}(z_{\text{slow}})]_0^1}_{\text{exclusion}} \quad (6)$$

$s_{\text{per}}(\cdot)$ scores reading the persistent factor from a subspace and $s_{\text{tra}}(\cdot)$ the transient proxy; $[\cdot]_0^1$ clips into $[0, 1]$. Classification scores already lie in this range, but R^2 goes negative when a prediction is worse than the mean, and the clipping acts only there, preventing a negative value from flipping the sign of the product. There is no normalization by any denominator anywhere.

Exclusion is the term that carries the claim. Unlike the other two it measures what is absent, and the objective enforces inclusion but not exclusion, so it is precisely the quantity we have to measure rather than assume.

Probe protocol. With the encoder frozen we fit only a linear classifier or regressor. Splits are per group (subject for HAPT, patient for PTB-XL, a front/back time split for the synthetic data), evaluation windows are non-overlapping to prevent leakage (asserted in the code), and only positions after the minimum context length (64 patches) are scored, since with less context the encoder lacks the evidence to read the persistent state. A random subspace of the same width is scored as a control.

F. Hand-Picked Values

Most knobs of this method cannot be chosen without labels. The cross-covariance weight, the block width, the target window, the patch size and the range of training horizons have no derivation, and we set them by looking at downstream performance; model selection without labels is a known difficulty in this literature [24]. Only τ is a partial exception: it can be initialized from the transient factor’s measured autocorrelation time, but that rule fails on signals with a regularly repeating component — on PTB-XL the per-record estimates spread over a factor of 499 and the rule’s output falls outside the training horizon range, so the value we use is an arbitrary choice rather than the rule’s output. One necessary condition we can state is that horizons must straddle the threshold, $\Delta_{\text{min}} < \tau < \Delta_{\text{max}}$, or the gate never opens or never closes. One quantity does not depend on labels: the exclusion term is measured from the transient proxy, which is computed

TABLE II. Separation index by mechanism. Cells are SEP (higher is better), $n = 5$ seeds, $\lambda = 4$. Bold marks the best cell in each row. The gate and the penalty are complementary: neither alone reaches what the two reach together.

Dataset	Stem	Neither	Gate only	xcov only	Gate + xcov
Synthetic	Reg	0.135	0.342	0.279	0.408
Synthetic	NCE	0.121	0.124	0.488	0.749
PTB-XL	Reg	0.375	0.404	0.312	0.495
PTB-XL	NCE	0.392	0.503	0.283	0.567
HAPT	Reg	0.521	0.537	0.236	0.632
HAPT	NCE	0.297	0.366	0.162	0.515

TABLE III. Latent target against raw target. Cells are SEP on synthetic, $n = 5$ seeds, each stem at its own best λ . Bold marks the latent target. Everything but the prediction target is held fixed.

Stem	Latent (best)	Raw (best)	Difference
HGLP-Reg	0.809 ($\lambda = 1$)	0.415 ($\lambda = 4$)	+0.394 (1.95 $\times$)
HGLP-NCE	0.844 ($\lambda = 16$)	0.432 ($\lambda = 16$)	+0.411 (1.95 $\times$)

directly from the signal, so a failed separation is detectable without task labels.

IV. Results

Part 1: Does the separation happen, and what creates it?

A. What creates the separation

We decompose the contribution of the two devices we add — the horizon gate and the cross-covariance penalty — by switching each on and off. The mechanism-free cell (Section III-D) is also the baseline in which only the coordinate split remains.

In Table II the model with both devices ranks first in all six settings, and neither device alone reaches that value. On the contrastive stem of the synthetic data the gate alone is indistinguishable from the mechanism-free case (0.124 against 0.121), yet adding that same gate to the penalty raises SEP from 0.488 to 0.749: the two devices need each other. The penalty alone is actively harmful on the real data, breaking down the inclusion term and falling below even the mechanism-free case.

The “Neither” cells take the first 16 of the 64 dimensions, and their SEP agrees in all six settings to within 0.015 with the SEP of 16 dimensions drawn at random from the same embedding. Without our devices, any part of the embedding holds the two factors to a similar degree, so coordinate position carries no meaning by itself; our contribution is to give those coordinates structure.

B. Separation requires a latent target

This section ablates a third design axis, the target space, testing the premise of the latent-prediction family: that prediction should be made in a latent space rather than on the raw signal. We hold the gate, the anchor, the horizon and the loss form fixed and change only what is predicted. The latent target is the 64-dimensional output the target encoder reads from the future interval; the raw target is the patch itself at that time (with one extra linear head). Both models carry the cross-covariance penalty, so λ has the same meaning for each, and we compare each at its own best value over the swept curve.

TABLE IV. Our method against label-free post-hoc unmixings. Cells are SEP, $n = 5$ seeds. Our method has one knob, so we report its best point on the swept λ curve; each unmixing is a single fixed value. Bold marks the winner of each row. The verdict is three to three overall: two of two on synthetic, one of four on real data.

Dataset	Stem	Ours (best λ)	SFA	Verdict
Synthetic	Reg	0.814 ($\lambda = 1$)	0.446	win
Synthetic	NCE	0.849 ($\lambda = 16$)	0.388	win
PTB-XL	Reg	0.495	0.567	loss
PTB-XL	NCE	0.567	0.599	loss
HAPT	Reg	0.677	0.717	loss
HAPT	NCE	0.626 ($\lambda = 64$)	0.527	win

Replacing the latent target with raw patches halves SEP on both stems (Table III); since both carry the same penalty and are compared at their own optimal λ , neither side gains from the knob. The allocation term is the same for the two targets, at 0.86 to 0.89, and the gap opens in the inclusion term: the raw target loses the persistent factor as the penalty grows, falling from 0.807 to 0.401, whereas the latent target stays above 0.97 throughout. The raw signal still contains the transient component, and the pressure to predict it too pushes out the persistent factor. The gate and the penalty of Section IV-A are thus not enough on their own; the latent target has to be present as well.

C. Can post-hoc processing take its place?

If an embedding trained without the gate is rotated post-hoc without labels — by principal component analysis, by slowness-ordered independent component analysis, or by slow feature analysis (SFA) — does it reach the same separation? These methods need no horizon gate at all, so they are the sharpest objection to our method.

We have one knob and the post-hoc rotations have none, so we report the whole curve rather than a single point (Table IV, Fig. 4); only our method has a knob that can be swept, and hiding that asymmetry would not make the comparison fair. On the synthetic data the curve passes above every post-hoc rotation point, and principal component analysis fails completely (SEP 0.000) — direct evidence that a ranking by variance is not a ranking by persistence, since the transient factor crowds into the leading components and nothing is left in the complement. On the real data we do not lead: SFA reaches the same inclusion at a higher exclusion. The verdict over all six settings is three wins to three losses, but split apart it is two of two on synthetic and three losses of four on real data, with losing margins of 0.032 to 0.072 against a winning margin of 0.099.

This shortfall is expected. Persistence and spectral slowness coincide only under particular assumptions, and our synthetic data deliberately breaks that coincidence: the persistent factor hides inside the oscillation frequency and cannot be reached by low-pass filtering. On PTB-XL and HAPT the persistent factor largely coincides with the slow component, so SFA, which optimizes slowness directly, does the same job by a more direct route. Our advantage appears where the two axes come apart; where

TABLE V. Degradation under domain shift: a difference of differences. Each cell is the drop in macro-F1 from the clean domain to the most perturbed one, for the gated block minus the same drop for a width-matched block of the mechanism-free model; paired by seed, $n = 5$. Negative means the gated block degrades less. These are differences of drops, not differences in absolute performance — the gated block does not overtake the mechanism-free one at any perturbation strength.

Setting	25 labels	100 labels	All labels
HAPT / Reg	-0.113 ± 0.023	-0.096 ± 0.029	-0.053 ± 0.056
HAPT / NCE	-0.017 ± 0.072	+0.015 ± 0.012	+0.007 ± 0.016
PTB-XL / Reg	-0.044 ± 0.048	-0.033 ± 0.048	-0.024 ± 0.035
PTB-XL / NCE	-0.036 ± 0.032	-0.010 ± 0.024	-0.009 ± 0.024
Synthetic / Reg	-0.012 ± 0.102	+0.043 ± 0.099	+0.064 ± 0.077
Synthetic / NCE	-0.044 ± 0.069	-0.055 ± 0.054	-0.045 ± 0.055

they coincide, post-hoc rotation already suffices. What remains ours is that the coordinates are fixed during training, so no second fitting stage is needed at deployment — an operational difference, not a difference in the quality of the separation.

Part 2: Practical effect and cost

D. The cost of separation, and stability under domain shift

We measure two things: the downstream cost that accompanies the separation, and how far performance falls when a probe fitted with few labels is carried to a domain in which the transient factor has been perturbed. What we measure is not the difference in absolute performance between the two blocks but the difference in the drop each block suffers; a smaller drop for the gated block means the separation translates into stability under domain shift.

We measure the cost against the full embedding of a model trained with no mechanism at all rather than against our own z_{full} , since the latter mixes in the effect of block width. Read that way, the cost is effectively zero on synthetic data and PTB-XL, and only HAPT shows a substantial value (-0.111 on the regression stem, -0.072 on the contrastive stem). HAPT is the only dataset in which the persistent factor is not constant within the window: 57% of its windows straddle an activity transition.

The perturbed domain is made by multiplying the evaluation windows by a gain ($x \mapsto x \cdot (1 + s)$, maximum perturbation $s = 2$); this is a bijection and does not destroy the labels of the persistent factor — even at maximum perturbation the HAPT activity labels survive at macro-F1 0.696 (chance 0.167). The encoder was trained on the clean domain alone, and the probe is fitted only on clean data with few labels (25 or 100); only the evaluation runs separately on clean and perturbed windows, so the probe never trains on the perturbation it is scored on.

The downstream cost of the gated block is effectively zero on synthetic and PTB-XL and at most 0.11 macro-F1 on HAPT. On that same HAPT the gated block has the smallest drop under shift: at 25 labels its drop is 0.113 smaller than that of the width-matched mechanism-free block (Table V), five times the seed standard deviation.

SEP vs λ . Our knob traces a curve (best ringed); post-hoc unmixings are flat lines. Above the line = win.

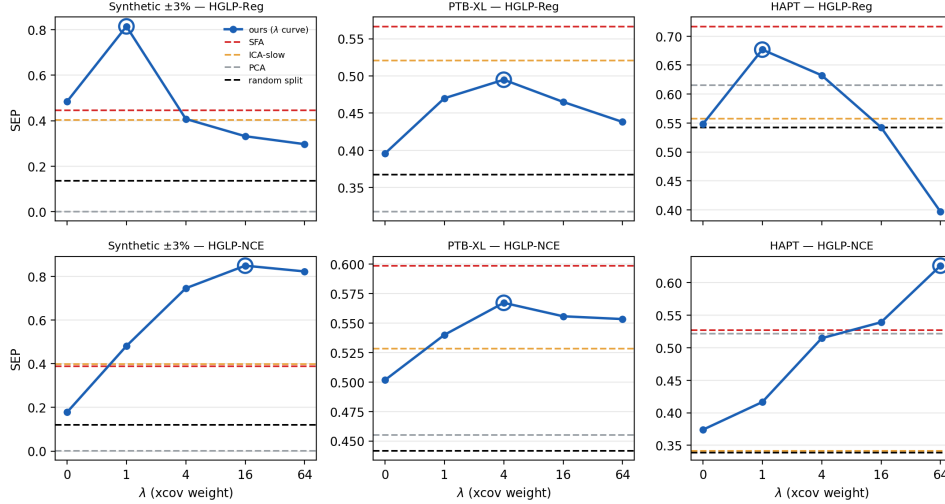


Fig. 4. SEP against the penalty weight λ . $n = 5$ seeds. Our method traces a curve over $\lambda \in \{0, 1, 4, 16, 64\}$ with its best point ringed; each post-hoc unmixing has no knob and is a horizontal line. A win is our curve rising above the SFA line. Unlike an inclusion–exclusion plane, this view combines all three SEP terms, so it also shows the one real-data win, which the inclusion–exclusion plane cannot display.

This is only a difference of drops. At no perturbation strength we tested does absolute performance overtake that model, and the effect exceeds the error bars in only one of the six settings, the HAPT regression stem; in the remaining five it lies within the noise, though by sign alone 17 of the 24 cells are negative.

The two results interlock: HAPT is both the dataset with the largest downstream cost and the only one with clear stability under domain shift, two sides of one phenomenon. The gate trades absolute performance for stability under domain shift; the two cannot be had together.

V. Conclusions

The horizon gate and the cross-covariance penalty split the persistent factor into a designated coordinate block, and across all six settings formed by three datasets and two loss families they jointly produce a separation that neither device reaches alone. The separation requires a latent target: switching only the prediction target to the raw signal halves the separation score, and what breaks down is precisely the inclusion of the persistent factor. On synthetic data we exceed label-free post-hoc rotation for both loss families, but on real data we fall short of Slow Feature Analysis, losing three of the four real settings, and the separation costs 0.111 macro-F1 downstream on HAPT. The gate trades absolute performance for stability under domain shift.

A. Relevance to Responsible AI

Explanation comes from the design, not after the fact. Most explainability techniques attach a story—saliency maps, attention visualizations—on top of an already trained entangled model. We instead put the axis of

interpretation inside the training objective, so z_{slow} is reserved for the persistent state before any probe is run, as measured by the block $\times$ factor matrix and by SEP. But the boundary is plain: the block structure is interpretable, the encoder is not. The latent space is rotation symmetric, so any 16 dimensions can hold the persistent factor; we assign no meaning to individual dimensions.

It degrades less where it matters. On HAPT, split by subject, z_{slow} drops the least when the domain is perturbed and labels are scarce—at 25 labels it loses 0.113 less macro-F1 than a width-matched mechanism-free block, five times the seed standard deviation. But that is degrading less, not being better: absolute performance never overtakes the mechanism-free embedding at any perturbation strength tested, and the effect exceeds the error bars in only one of six settings.

The cost becomes a measurable quantity. The design makes the downstream cost measurable: because the method explicitly produces a block holding the persistent state, the cost of solving a task from that block alone is a single number—the macro-F1 difference against the full embedding z_{full} of a model trained with no mechanism at all. With an entangled representation there is no such quantity to measure.

B. Limitations

First, the hyperparameters cannot be chosen without labels. The cross-covariance coefficient, the block width, the target width, the patch size, and the horizon range have no derivation rule and were set by downstream performance. The autocorrelation-derived horizon threshold is a partial exception, but it fails on regularly repeating signals: on PTB-XL the per-record estimates scatter from 0.1 to 49.9 patches and the rule returns 56.6, outside the

training horizon range, so we set the threshold by hand instead. The exclusion term needs no task labels, so a failed separation can at least be detected label-free.

Second, we did not analyze the target width. We used 8, 12, and 8 patches, but the ratio to the transient factor’s autocorrelation time is 1.53, 0.91, and 0.42—not constant. This width governs how much of the transient factor is averaged away inside the target; the sweep was not run.

Third, we did not demonstrate a practical benefit: the perturbed-domain effect clears the error bars in only one of six settings, and even there absolute performance never overtakes the mechanism-free embedding.

Fourth, the claim that a factor can be “persistent yet absent from the slow part of the signal” rests on a single synthetic dataset. One counterexample refutes a universal proposition, but we designed it ourselves, and neither PTB-XL nor HAPT exhibits the case.

Fifth, our axis selects persistence, not relevance. A persistent nuisance factor—the low-frequency drift respiration and electrode motion produce in an electrocardiogram—enters z_{slow} by the same criterion, and which persistent factors matter remains a human judgment.

C. Future Work

We compared only against label-free post-hoc rotation, so methods that place factor separation explicitly in the objective remain to be compared. Validation is also needed on data whose persistent factor varies within a recording while staying constant within a window — PTB-XL fails the first property and HAPT the second, whereas sleep-stage classification has both. Finally, using the trajectory of the separated block as a state indicator, rather than as classifier input, would address the fourth limitation directly.

References

- [1] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” arXiv:1807.03748, 2018.
- [2] Y. LeCun, “A path towards autonomous machine intelligence,” v0.9.2, OpenReview, 2022.
- [3] M. Assran et al., “Self-supervised learning from images with a joint-embedding predictive architecture,” in CVPR, 2023, pp. 15619–15629.
- [4] W.-N. Hsu, Y. Zhang, and J. Glass, “Unsupervised learning of disentangled and interpretable representations from sequential data,” in NeurIPS, 2017.
- [5] Y. Li and S. Mandt, “Disentangled sequential autoencoder,” in ICML, 2018.
- [6] R. B. Cleveland et al., “STL: A seasonal-trend decomposition procedure based on Loess,” J. Official Statistics, vol. 6, no. 1, pp. 3–73, 1990.
- [7] N. E. Huang et al., “The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis,” Proc. R. Soc. Lond. A, vol. 454, pp. 903–995, 1998.
- [8] A. Hyvärinen and H. Morioka, “Unsupervised feature extraction by time-contrastive learning and nonlinear ICA,” in NeurIPS, 2016.
- [9] A. Khinchin, “Korrelationstheorie der stationären stochastischen Prozesse,” Math. Annalen, vol. 109, pp. 604–615, 1934.
- [10] W. Bialek, I. Nemenman, and N. Tishby, “Predictability, complexity, and learning,” Neural Comput., vol. 13, no. 11, pp. 2409–2463, 2001.
- [11] F. Creutzig and H. Sprekeler, “Predictive coding and the slowness principle: An information-theoretic approach,” Neural Comput., vol. 20, no. 4, pp. 1026–1041, 2008.
- [12] L. Wiskott and T. J. Sejnowski, “Slow feature analysis: Unsupervised learning of invariances,” Neural Comput., vol. 14, no. 4, pp. 715–770, 2002.
- [13] T. Blaschke, T. Zito, and L. Wiskott, “Independent slow feature analysis and nonlinear blind source separation,” Neural Comput., vol. 19, no. 4, pp. 994–1021, 2007.
- [14] S. Richthofer and L. Wiskott, “Predictable feature analysis,” arXiv:1311.2503, 2013.
- [15] R. B. Randall and J. Antoni, “Rolling element bearing diagnostics — A tutorial,” Mech. Syst. Signal Process., vol. 25, no. 2, pp. 485–520, 2011.
- [16] I. Higgins et al., “ β -VAE: Learning basic visual concepts with a constrained variational framework,” in ICLR, 2017.
- [17] Y. Nie et al., “A time series is worth 64 words: Long-term forecasting with transformers,” in ICLR, 2023.
- [18] Z. Yue et al., “TS2Vec: Towards universal representation of time series,” in AAAI, vol. 36, 2022, pp. 8980–8987.
- [19] S. Tonekaboni, D. Eytan, and A. Goldenberg, “Unsupervised representation learning for time series with temporal neighborhood coding,” in ICLR, 2021.
- [20] S. Ennadir, S. Golkar, and A. Sarra, “Joint embeddings go temporal,” arXiv:2509.25449, 2025.
- [21] A. Chemeris, M. Jin, and R. Balestrieri, “LeNEPA: No-augmentation next-latent prediction for time-series representation learning,” arXiv:2607.00958, 2026.
- [22] J. Petersen et al., “HEPA: A self-supervised horizon-conditioned event predictive architecture for time series,” arXiv:2605.11130, 2026.
- [23] D. A. Klindt et al., “Towards nonlinear disentanglement in natural data with temporal sparse coding,” in ICLR, 2021.
- [24] F. Locatello et al., “Challenging common assumptions in the unsupervised learning of disentangled representations,” in ICML, 2019.